

Forwards, Discounting and Dividends

Parity-implied forwards, a robustness clamp, and American de-bias

Vol-Fitter Technical Notes, No. 06

Vol-Fitter

July 3, 2026

Abstract

Every smile in the Vol-Fitter is forward-normalized: log-moneyness is $k = \log(K/F_T)$, and a wrong forward shifts the entire smile, gapping it at the money. This note documents how the forward and discount are obtained and made robust. The primary route is a put–call parity regression: since $C(K) - P(K) = D(F - K)$, a straight-line fit of the call–put price difference against strike yields the discount D as its slope and the forward F from its intercept, iterated with outlier rejection. The parity *slope* is the worst-identified parameter, and on a noisy or stale feed it drifts to an unphysical rate that tilts the whole forward; we describe the *discount clamp* that pins the implied rate to a physical band and re-derives the forward from the well-identified price level when it bites — leaving clean chains untouched. Dividends enter as a discrete escrowed cash schedule or a continuous yield, and a coarse *American de-bias* iterates the carry to the fixed point that reconciles the de-Americanized puts and calls, with the discount held. A worked example recovers a known forward to the cent and shows the clamp catching a noisy slope.

Contents

1	Why the forward matters	2
2	The parity regression	2
3	The discount clamp	3
4	Dividends	4
5	American de-bias	4
6	What is original here	5
A	Hyperparameter atlas	5
B	Reference implementation: the discount clamp	5

1 Why the forward matters

The forward F_T is the anchor of the whole smile. Log-moneyness, the ATM point, the put/call OTM split, the de-Americanization (Note 05) and the var-swap replication (Note 08) are all defined relative to it. A forward error of even a fraction of a percent shifts the ATM strike and tilts the put–call balance, which shows up as a *gap* in the at-the-money smile — the most visible symptom of a bad forward. So the forward must be both *accurate* on clean data and *robust* on the noisy, delayed feeds the app often runs against.

Heuristic

The forward is not quoted; it is *inferred* from the option prices themselves through put–call parity. That makes it a fitted quantity with its own error bars, and the discount (the parity slope) is the least reliable of them. The whole design below is about extracting the forward where the data is informative (the price level) and refusing to trust it where it is not (the slope on a stale feed).

2 The parity regression

Put–call parity in forward-normalized, discounted terms reads

$$C(K) - P(K) = D(F - K), \quad (1)$$

where $D = e^{-rT}$ is the discount factor and F the forward. This is a straight line in K : regressing the call–put difference on strike,

$$\underbrace{C(K) - P(K)}_y = \underbrace{DF}_a - \underbrace{D}_{-b} K, \quad D = -b, \quad F = \frac{a}{D}, \quad (2)$$

gives the discount as the (negated) slope and the forward as the intercept divided by the discount. In code it is a one-line least squares (this recovers a known forward/discount to $10^{-6}/10^{-9}$ and equals the production regression to 10^{-9}):

Listing 1: Put–call parity regression for the forward and discount (2) (distilled from `data/forwards.py`).

```
1 def implied_forward(K, call_mid, put_mid):
2     y = call_mid - put_mid                # C - P = D F - D K = a + b K
3     b, a = np.polyfit(K, y, 1)            # slope b = -D, intercept a =
        D F
4     D = -b                                # discount factor
5     return a/D, D                          # forward F = a/D
```

The fit iterates: fit, measure residuals, drop paired strikes with implausible residuals (or fewer than three pairs), and refit, so a few crossed or stale quotes do not corrupt the line. Figure 1 shows the recovery on a clean synthetic chain — the forward comes back to the cent and the rate to its true 4%.

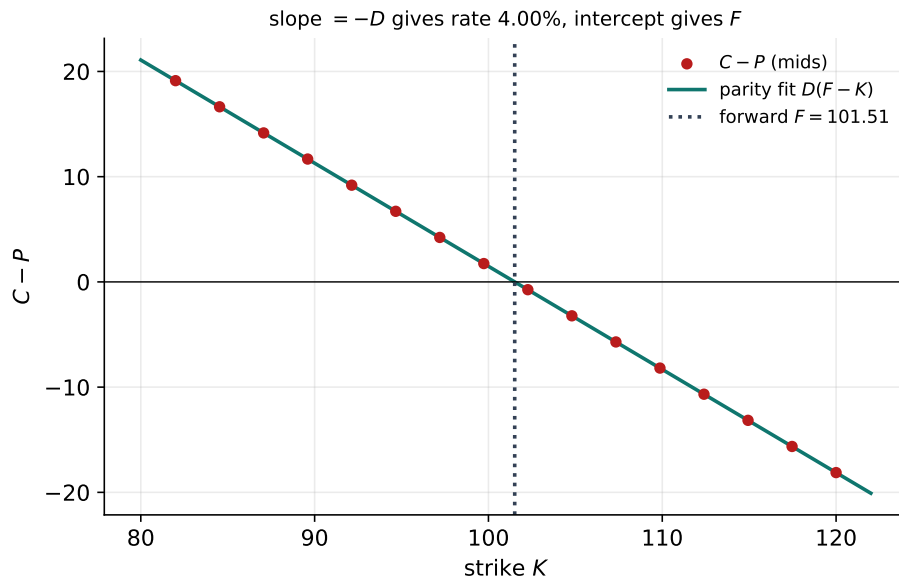


Figure 1: The put–call parity regression $C - P = D(F - K)$. The slope is the discount ($\rightarrow$ rate 4.00%), the intercept gives the forward ($F = 101.51$, true 101.51).

3 The discount clamp

The parity slope *is* the discount, and it is the worst-identified parameter in (2): the price difference $C - P$ changes slowly with strike, so a little noise in the mids tilts the line a lot. On a clean chain this is harmless; on a noisy or stale delayed feed it drifts to nonsense — an implied discount $D > 1$, i.e. a *negative* rate, which tilts the forward and gaps the ATM smile.

Caution

A negative implied rate is not a market signal here; it is feed noise in the least reliable regression parameter. Letting it through propagates a small price error into a large level error: a 5% discount error shifts the whole implied-vol level. The forward’s *level* (the intercept) is well identified; its *slope* (the discount) is not.

The fix, when a reference date pins the year fraction, is to clamp the parity-implied rate $r = -\log D/T$ to a physical band $[-5\%, 30\%]$. Clean chains sit strictly inside the band and are byte-identical. When the clamp bites, the forward is *re-derived from the well-identified price level* at the clamped discount, rather than from the tilted slope — so the robust forward keeps the good information (the level) and discards the bad (the runaway slope). A final sanity bound $|\log(F/S)| < \text{FWD_CLAMP_LOG} = 0.5$ falls back to spot if even the level is unusable. Figure 2 shows the implied rate leaving the band as the slope is perturbed and the clamp pinning it.

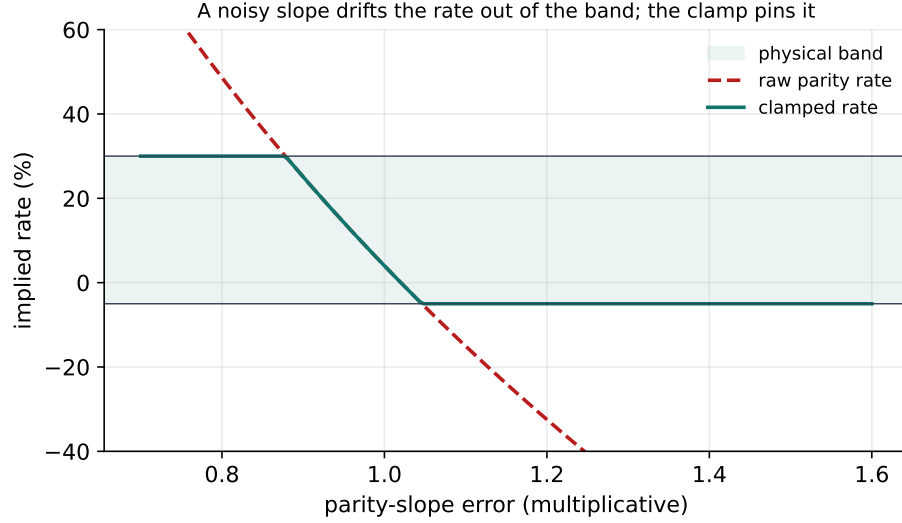


Figure 2: As a multiplicative slope error perturbs the discount, the raw parity rate (dashed) leaves the physical band; the clamp (solid) pins it to $[-5\%, 30\%]$ and the forward is re-derived from the price level.

4 Dividends

The carry that links spot to forward, $F = S e^{(r-q)T}$, must account for dividends. Two treatments coexist (Note 05 uses the same machinery for the pricing tree):

- **Discrete cash** (`discrete_absolute`): a schedule of ex-dates and cash amounts, priced by the escrowed-spot method — the present value of dividends in $(0, T]$ is stripped from the spot to form the diffusing lattice, and early exercise is evaluated against the actual cum-dividend spot.
- **Continuous yield** (`continuous`): a constant q , a crude but cheap proxy for far-dated proportional dividends, and a coexisting tail when combined with a discrete near schedule.

The dividend mode is per ticker, editable, and feeds both the forward carry and the de-Americanization tree.

5 American de-bias

There is a circularity between the forward and de-Americanization: the forward sets the carry q , the carry drives early exercise, and de-Americanizing the puts and calls re-implies the forward. The raw parity regression uses American mids directly, which biases the forward slightly because the early-exercise premium is not symmetric between puts and calls. The *de-bias* resolves this with the discount *held*: fixing the rate $r = -\log D/T$ from the raw parity, each round sets the carry $q = r - \log(F/S)/T$ from the current forward, de-Americanizes the near-ATM call and put mids to European-equivalent Black vols, reprices them as European, and re-implies the forward at the *fixed* slope $-D$,

$$F \leftarrow \frac{1}{D} \overline{(y + D K)}, \quad y = \text{European } (C - P), \quad (3)$$

iterating to the fixed point where the two de-Americanized OTM sides join. Only a near-ATM band of strikes and a coarse tree enter — it locates the forward to ~ 1 basis point in a few milliseconds, while quote prep does the full-precision per-quote de-Americanization downstream. The discount is left exactly as the raw parity gave it.

6 What is original here

Parity forward extraction is textbook; the robustness engineering is the contribution. The *discount clamp* encodes the right epistemics — trust the well-identified price level, distrust the ill-identified slope, and re-derive the forward accordingly — so the app survives the noisy delayed feeds that would otherwise gap every ATM smile. The *de-bias* closes the forward/de-Am circularity at the fixed point that quote prep actually uses, with the discount held so the two corrections do not fight. Both are byte-identical on clean data, so the robustness costs nothing when it is not needed.

A Hyperparameter atlas

Table 1: Forward/dividend hyperparameters.

Knob	Default	Role
<i>Surfaced</i>		
<code>ForwardPolicy.mode</code>	<code>parity</code>	<code>parity</code> (regression) or <code>theoretical</code> (carry model).
<code>MarketSettings.rate r</code>	—	Risk-free rate for the theoretical forward / tree.
<code>dividendMode</code>	<code>continuous</code>	<code>continuous</code> yield or <code>discrete_absolute</code> cash.
<code>dividends</code>	empty	Discrete (ex-date, amount) schedule.
<i>Hidden</i>		
<code>RATE_MIN/MAX</code>	<code>[-5%, 30%]</code>	Physical band for the parity-implied rate (clamp).
<code>FWD_CLAMP_LOG</code>	<code>0.5</code>	Sanity bound on $ \log(F/S) $; else fall back to spot.
<code>MIN_PAIRED_STRIKES</code>	<code>3</code>	Minimum paired strikes for a parity fit.
<code>DEAM_REFINE_BAND</code>	<code>near-ATM</code>	Strike band entering the de-bias iteration.
<code>MAX_DEAM_ITERS</code>	<code>few</code>	De-bias fixed-point iterations.

Performance. The parity regression is a 2×2 least squares per expiry; the clamp is a comparison; the de-bias is a handful of coarse-tree de-Americanizations on a near-ATM band (~ 1 bp forward in a few ms). All are negligible against the smile fit, and editing a name’s rate or dividend schedule bumps only that ticker’s forwards version, refitting just that name.

B Reference implementation: the discount clamp

The parity *slope* is the discount — the worst-identified parameter — so on a noisy feed it can imply a nonphysical rate. The clamp pins the rate to a band and, when it bites, re-derives the

forward as a per-strike average $F = \text{mean}(K + (C - P)/D)$ (the well-identified *level*) rather than the slope-driven a/b , so the forward no longer inherits the bad slope.

Listing 2: The discount clamp and forward re-derivation (distilled from `data/forwards.py`).

```
1 def clamp_forward(K, call_mid, put_mid, t, F, D, r_lo=-0.05, r_hi
   =0.30):
2     r = -np.log(D)/t                                # the parity discount's
   implied rate
3     if r_lo <= r <= r_hi:
4         return F, D                                # physical: keep the
   regression result
5     D = np.exp(-np.clip(r, r_lo, r_hi)*t)           # clamp the rate to
   the band
6     F = np.mean(K + (call_mid - put_mid)/D)         # re-derive F from
   the LEVEL, not a/b
7     return F, D
```

References

- [1] J. Hull. *Options, Futures, and Other Derivatives*. Pearson, 10th ed., 2017.
- [2] J. Gatheral. *The Volatility Surface*. Wiley, 2006.